

带权并查集

人员

王承周、许睿谦、阮文璋、王毅博 到课

上周作业检查

上上周作业链接: <https://vjudge.net/contest/807399>

概览 题目 提交状态 排名 (14:05:43:59) 讨论									
Rank	Team	Score	Penalty	A 9 / 15	B 6 / 11	C 6 / 10	D 4 / 7	E 3 / 12	
1	☆  yyyjc	5	72461	9:01:52:22	9:01:12:49 (-1)	9:01:48:24	10:16:13:31	12:08:54:49 (-4)	
2	☆  r111	5	82067	9:06:09:00	10:12:08:00	11:12:49:00	11:13:37:00 (-3)	14:02:04:00	
3	☆  lyc12345bc (刘奕辰)	5	91443	13:14:26:36 (-1)	9:00:53:07	13:13:02:00 (-2)	13:14:08:51	13:15:13:00 (-4)	
4	☆  wcz123bc	4	71455	9:05:49:12	13:10:45:00	13:11:21:00	13:11:00:00		
5	☆  junyan007008	3	41174	9:07:03:00 (-2)	9:06:50:00	9:23:01:00 (-2)		(-1)	
6	☆  lijinsu	3	52161	13:12:43:00 (-1)	9:01:31:00 (-4)	13:13:27:00			
7	☆  chenxinmiao (陈欣妙)	1	13308	9:05:48:46					
8	☆  wyb1111 (王毅博)	1	13310	9:05:50:00					
9	☆  xuruiqian	1	13375	9:06:15:00 (-2)					

上周作业链接: <https://vjudge.net/contest/810246>

本周作业

<https://vjudge.net/contest/811695> (课上讲了本周的 A B D 题, 课后作业是本周的 C 题)

课堂表现

今天着重带同学们复习了之前的几个背包, 并讲解了带权并查集的知识点, 同学们课上没时间写 D 题, 课后要把这道题补上

课堂内容

宝物筛选

多重背包二进制版, 用二进制进行优化

比如说有 100 个某物品, 可以认为有 1个、2个、4个、8个、16个、32个、37个 这些组

```
#include <bits/stdc++.h>

using namespace std;

const int maxn = 4e4 + 5;
int f[maxn];

int main()
{
    int n, W; cin >> n >> W;
    while (n -- ) {
        int v, w, m; cin >> v >> w >> m;
        int sum = 0;
        for (int i = 1; sum+i <= m; i *= 2) {
            for (int j = W; j >= i*w; --j) f[j] = max(f[j], f[j-i*w]+i*v);
            sum += i;
        }
        m -= sum;
        for (int j = W; j >= m*w; --j) f[j] = max(f[j], f[j-m*w]+m*v);
    }
    cout << f[W] << endl;
    return 0;
}
```

通天之分组背包

```
#include <bits/stdc++.h>

using namespace std;

const int N = 1000 + 5, M = 100 + 5;
struct node {
    int c, v;
};
vector<node> vec[M];
int f[N];

int main()
{
    int n, m; cin >> m >> n;
    for (int i = 1; i <= n; ++i) {
        int c, v, id; cin >> c >> v >> id;
        vec[id].push_back({c,v});
    }

    for (int i = 1; i <= 100; ++i) {
        for (int j = m; j >= 0; --j) {
            for (node it : vec[i]) {
```

```

        if (j >= it.c) f[j] = max(f[j], f[j-it.c]+it.v);
    }
}
}
cout << f[m] << endl;
return 0;
}

```

CF148E Porcelain

可以先预处理, 求出来每一行分别选 1 个, 2 个, 3 个, ... 时能获得的最大价值

接下来, 就是在每一行挑几个, 问正好选够 m 个时, 能获得的最大价值是多少, 问题转化为了一个分组背包的板子题

```

#include <bits/stdc++.h>

using namespace std;

const int N = 100 + 5, M = 10000 + 5;
int len[N], w[N], pre[N], c[N][N], f[M];

int get_sum(int l, int r) { return pre[r] - pre[l-1]; }

void solve(int id) {
    cin >> len[id];
    for (int i = 1; i <= len[id]; ++i) cin >> w[i], pre[i] = pre[i-1]+w[i];

    for (int i = 1; i <= len[id]; ++i) {
        for (int j = 0, k = i; j <= i; ++j, --k) {
            c[id][i] = max(c[id][i], pre[j] + get_sum(len[id]-k+1, len[id]));
        }
    }
}

int main()
{
    int n, m; cin >> n >> m;
    for (int i = 1; i <= n; ++i) solve(i);

    for (int i = 1; i <= n; ++i) {
        for (int j = m; j >= 0; --j) {
            for (int k = 1; k <= len[i]; ++k) {
                if (j-k >= 0) f[j] = max(f[j], f[j-k]+c[i][k]);
            }
        }
    }
    cout << f[m] << endl;
    return 0;
}

```

[NOI2001] 食物链

f[i]: 维护 i 的父亲

d[i]: 维护 i 与 i 的父亲 之间的距离

路径压缩代码:

```
int fFind(int x) {
    if (f[x] != x) {
        int p = f[x];
        f[x] = fFind(f[x]);
        d[x] = (d[x] + d[p]) % 3;
    }
}
```

```
#include <bits/stdc++.h>

using namespace std;

const int maxn = 5e4 + 5;
const int mod = 3;
int f[maxn], d[maxn];

int fFind(int x) {
    if (f[x] != x) {
        int p = f[x];
        f[x] = fFind(f[x]);
        d[x] = (d[x] + d[p]) % mod;
    }
    return f[x];
}

int main()
{
    int n, k, res = 0; cin >> n >> k;
    for (int i = 1; i <= n; ++i) f[i] = i;

    while (k -- ) {
        int op, x, y; cin >> op >> x >> y;
        if (x>n || y>n) { ++res; continue; }

        int p = fFind(x), q = fFind(y);
        if (p == q) {
            if (op==1 && d[x]!=d[y]) ++res;
            if (op==2 && d[x]!=(d[y]+1)%mod) ++res;
        } else {
            f[p] = q, d[p] = (d[y]+(op-1)-d[x]+mod) % mod;
        }
    }
}
```

```
cout << res << endl;  
return 0;  
}
```